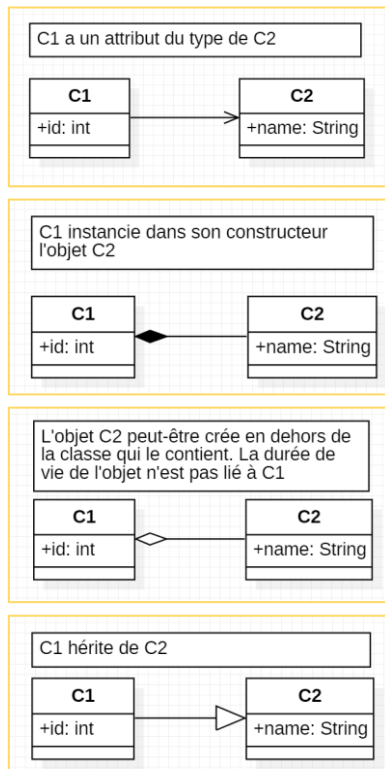


- Le MCD est le modèle conceptuel des données.
- C'est-à-dire les entités concernées sous forme de classes n'implémentant aucune méthode mais avec les attributs nécessaires pour les décrire entièrement.
- Dans le MCD, on retrouve les entités et les relations entre elles : on se concentre sur les relations et les cardinalités. Pas les méthodes



• Objectif d'une architecture logique

- Encapsuler et décomposer la complexité
- Faciliter le travail en équipes
- Faciliter la réutilisation et l'évolutivité
- **Forte cohésion** intra paquetage
- **Faible couplage** inter paquets

Cohésion : Si les classes dans un même package ont des points communs.

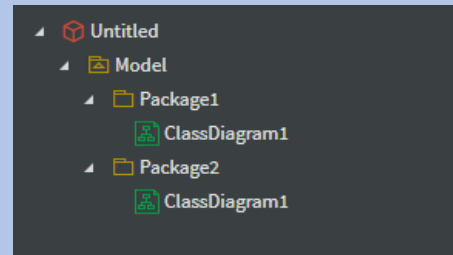
- Points communs : forte cohésion 😊
- Peu de points communs : faible cohésion

Couplage : Détermine le nombre de connexion entre package

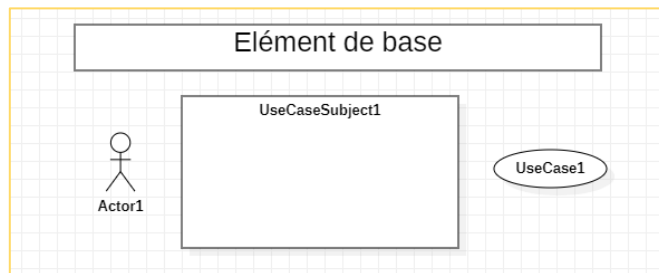
- Bcp de connexion : couplage fort
- Peu de connexion : couplage faible 😊

MCD (Conceptuel)

DIAGRAMMES DE CLASSES. DOCUMENTATION.



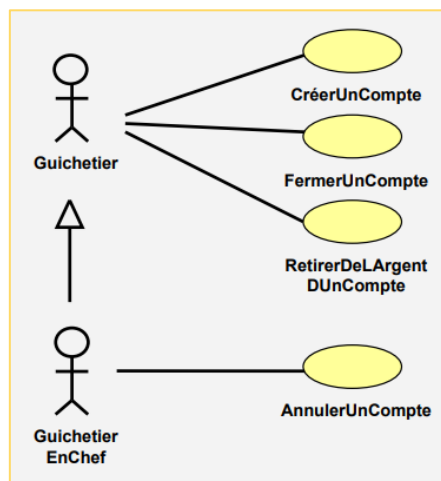
- Le MCT est le modèle conceptuel des traitements. Il montre ce que l'acteur peut faire comme action.
- On décrit les acteurs et leurs actions au sein d'un système.
- Le diagramme de UC permet de décrire les fonctionnalités du système ainsi que définir ses limites.



• Acteurs

- Vivant ou non vivant
- Placer à gauche pour les principaux
- Placer à droite pour les secondaires
- Peut avoir plusieurs rôles
- Pas de relation entre acteur – acteur

Généralisation :

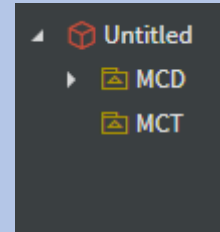


« Le chef peut effectuer toutes les actions d'un guichetier + ses actions »

MCT

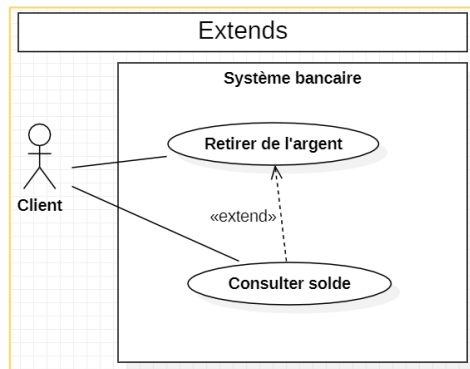
(Conceptuel)

DIAGRAMMES DE USE CASE (UC). UC DOCUMENTATION.

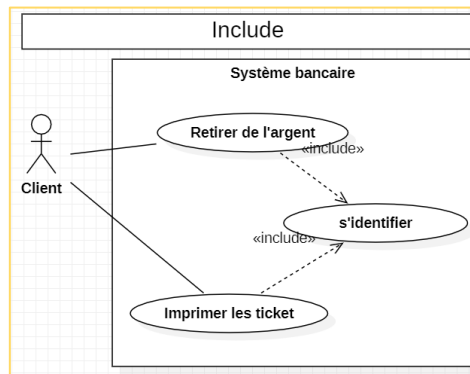


• Cas d'utilisation

- Doit être concis et claire
- Peut-être relier à plusieurs acteurs
- Doit-être utile
- Décris les buts, les besoins des acteurs et les interactions

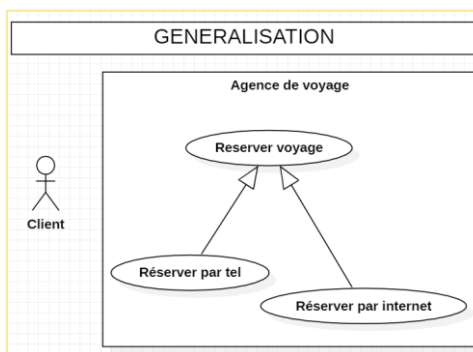


« Consulter son solde permet de de retirer de l'argent »



« Retirer de l'argent engendre une instance de s'identifier »

« Si on à retirer de l'argent c'est qu'on s'est identifier »



« On peut réserver un voyage par tel ou par internet »

Dans la figure de l'include, un client peut retirer de l'argent, et peut imprimer les extraits, ces deux actions appel le use case identifier obligatoirement.

- **Niveau d'abstraction**

- **Cloud level** : trop peu détaillé.
- **Kite level** : un peu plus détaillé (niveau résumé).
- **Sea level** : décrit un but de l'acteur qu'il peut atteindre via une interaction avec le système.
- **Fish level** : décrit une interaction avec le système, pas un but en soi.
- **Clam level** : bcp trop de détail.

- **UC doit respecter les 4 unicités**

- **Unicité d'objectif.**

Le UC doit fournir un résultat appréciable à l'acteur avec lequel il interagit.

- **Unicité de responsabilité.**

Pour un UC interactif, un seul acteur direct

- **Unicité de temps.**

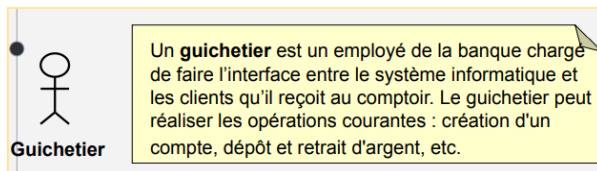
Unicité d'exécution Un enchaînement d'actions qui se suivent sans interruption

Unicité de périodicité Exécution à la même fréquence

- **Unicité de mode.**

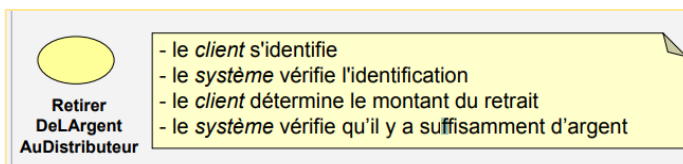
Manuel, interactif ou automatisé unitaire ou par lot

- **Description des acteurs**



- **Description brève des UC**

- Ce concentre sur le **scénario nominal**



- **Définition**

- Définir tous les détails d'un UC

- **Gabarits de la documentation**

#Description brève

[Définir en quelques phrase ce que fait ce Use Case]

- Son objectif
- L'acteur responsable
- Les circonstances dans lesquelles le UC se déroule
- L'essentiel du déroulement du UC

#Mode

- Interactif : Si l'acteur à un échange physique
- Automatiser : Aucune intervention n'est nécessaire

#Préconditions

- Décrire quel conditions l'utilisateur doit remplir pour effectuer ce UC.

#Postconditions

- Décrire le résultat du use case

#Flux de base

- Description du déroulement du use case dans le **scénario « normal »**.

#Flux alternatifs

- Scénario du use case autres que le flux de base : exceptions, problèmes, etc...

[Titre du flux alternatif 1]

[Écrire les numéros des actions et le cheminement]

[Titre du flux alternatif 2]

...

#Règle de calcul

- Décrire le calcul que fait le use case

#Exigences particulière

- [Hardware, performances]

UC SPECIFICATIONS

(Fonctionnel)

DOCUMENTATION DU UC.

PTFE

(Fonctionnel)

TESTS FONCTIONNELS ELEMENTAIRES.

- **2 Types de tests**

- White box(ouverte)
- Black box(fermée)

- **White box**

- On a accès au code source
- On vérifie le code (erreur, lecture, syntaxe)
- Géré par les informaticiens

- **Black box**

- Pas accès au code source
- On définit des scénarios et les résultats attendus
- On teste chaque scénario sur le logiciel développé
- On **compare les résultats obtenus vs attendus**
- Si test OK : validé.
- Si test KO : il faut corriger

- **Que tester ?**

Test fonctionnels (MCT & UC SPECIFICATION)

- Teste la **fiabilité** et **stabilité** du logiciel.
- Test de **pertinence** : évaluer si le logiciel répond de manière adéquate aux besoin spécifiés.

Tests de convivialité (ergonomie)

- **Look** : Evaluer l'apparence visuelle de l'interface utilisateur
- **Feel** : Evaluer la réactivité dynamique de l'interface utilisateur

Tests de performance

- Volumes de données, charges.

Tests de robustesses

- Charge le programme à fond pour voir quand il cède.

- Maintenant on implémente les **associations dirigées**

Associations dirigées

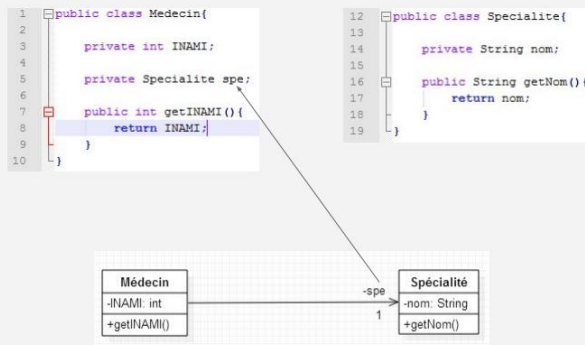
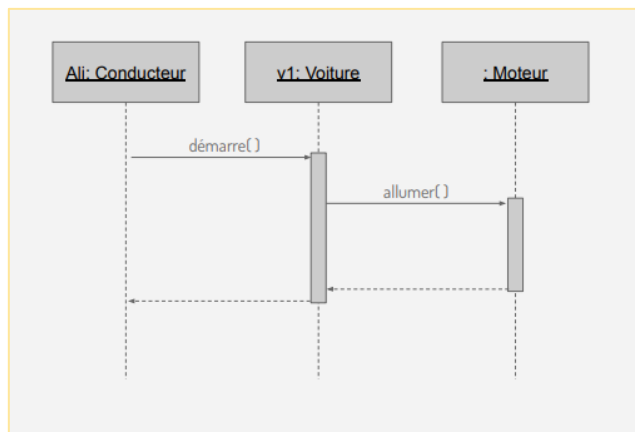


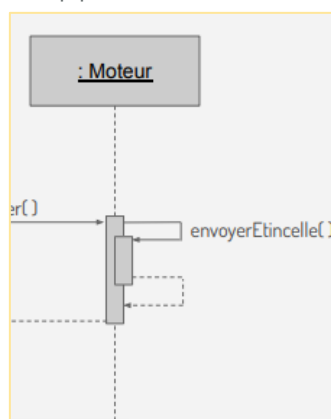
Diagramme de séquence



- Implémente une ligne de vie.
- Implémente des messages et activations

Quoi comprendre ci-dessus ?

- L'objet Voiture est instancié dans les attributs de Conducteur
- La méthode démarre () appartient à voiture
- Conducteur appelle la méthode démarre.
- La méthode démarre contient sans ça méthode un appel à la méthode allumer()



« Le Moteur à deux méthodes, `allumer()` et `envoyerEtincelles()`, `envoyerEtincelles` est appelé dans la fonction `allumer()` et agit que dans sa classe. »

MTD & MTT

(Technique)

DIAGRAMMES DE CLASSES TECHNIQUE. & DIAGRAMME DE SEQUENCE.

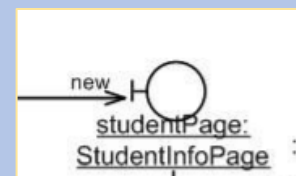
Visualisation code java MTT



Correction diagramme client
commande



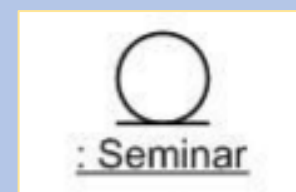
Vue



Controller



Objet

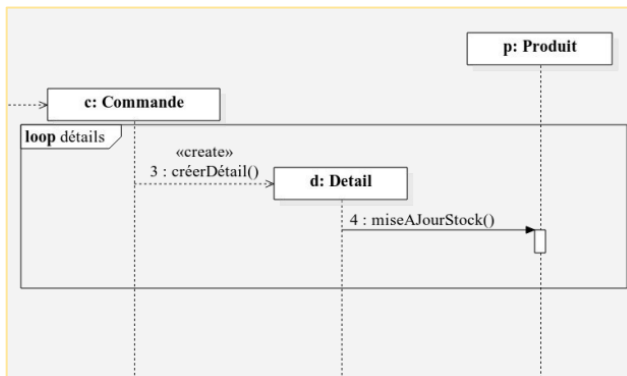


- **Controller**

- Reçois les éléments système
- S'occupe des tâches de contrôler et de coordination (séquence des appels)

- **Haute cohésion**

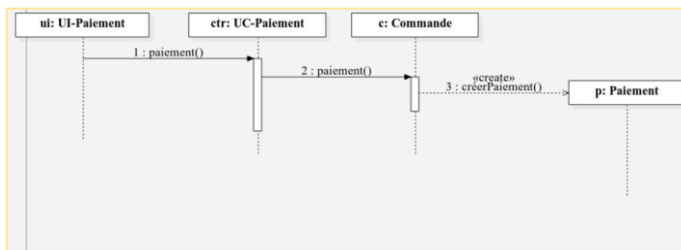
- Comment concevoir des objets clairs, gérables et responsable de leur tâche ? : **Maximiser la cohésion.**



« Avec la haute cohésion les objets ont un lien étroit »

- **Couplage**

- Comment réduire l'impact du changement dans le code ? : **Minimiser le couplage**



« Ainsi les dépendances sont correctement équilibrées »

- **Polymorphisme**

- Comment gérer les traitements alternatifs bases sur le type ? : **Assigner les responsabilités aux classes pour lesquelles le traitement varie.**
- Ne pas faire de « if » sur le type de l'objet.

UCR

(Technique)

DESIGN PATTERN MVP.

Problème du couplage fort :

Un changement dans une classe force à changer toutes ou la plupart des classes liées.

Modularité :

La modularité est la propriété d'un système qui a été décomposé en un ensemble de modules cohésifs et faiblement couplés.

KISS :

Coupez les codes complexe en morceau.

DRY :

Si un code se répète, faites-en une méthode commune, un héritage.

YAGNY :

N'écrivez pas du code qui ne servira pas ou pas maintenant.

DESIGN PATTERN

(Technique)

- **Définition : Design patterns**

- Décrivent une structure récurrente de design

- **Avantages : Design patterns**

- Montrent des solutions efficaces à des problèmes récurrent de dev.
- Améliore la qualité du code.
- Permet la réutilisation d'architecture logicielles.

- **4 éléments : Design patterns**

- Le nom du pattern
- La description du problème (quand appliquer le pattern)
- La description de la solution
- Les conséquences

- **3 types de design patterns**

- **Création**

(Focalise sur le processus de création d'objets)

- **Structure**

(Focalise sur la composition de classes pour former de plus larges structures)

- **Comportement**

(Focalise sur les algorithmes et l'attribution des responsabilités entre les objets)

- **2 niveaux (scope)**

- **Classe**

(Focalise sur les relations entre classes et sous-classes)

- **Objets**

(Focalise sur les relations entre objets qui peuvent changer durant l'exécution)

- **Design patterns : Singleton**

- Il est utilisé pour garantir qu'une classe n'a qu'une seule instance et fournit un moyen global d'y accéder.

- **Singleton : caractéristiques**

- **Instance unique**

Le Singleton garantit qu'une seule instance de la classe est créée et fournit un point d'accès global à cette instance.

- **Accès global**

Toutes les parties du programme qui ont besoin de l'instance unique peuvent y accéder à travers une méthode statique ou une propriété statique.

- **Construction unique**

La construction de l'instance unique est généralement gérée à l'intérieur de la classe Singleton, et l'accès à la construction est contrôlé pour éviter la création accidentelle de plusieurs instances.

- **Lazy loading**

L'instance unique n'est généralement créée que lorsque cela est nécessaire (appelé lazy loading), ce qui peut améliorer les performances en évitant une création inutile si l'instance n'est jamais utilisée.

Exemple : ²

```
public class Singleton {
    // Instance unique
    private static Singleton instance;

    // Constructeur privé pour empêcher la création directe d'instance
    private Singleton() {
        // Initialisation de l'instance
    }

    // Méthode statique pour obtenir l'instance unique (avec lazy load)
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    // Autres méthodes et propriétés de la classe Singleton
}
```

²La classe singleton n'est qu'un exemple, elle peut être remplacée par exemple par une classe **Point**, **qui va implémenter la même logique**.

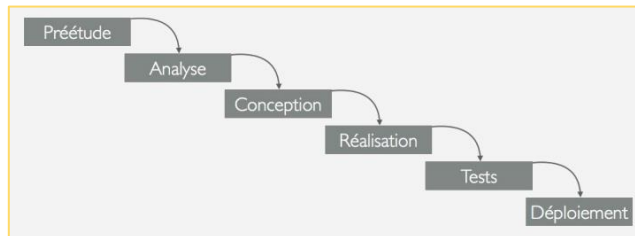
L'observer est déjà vu.

Composite : est utilisé pour traiter les collections d'objets de la même manière que les objets individuels, ce qui facilite la manipulation des structures hiérarchiques complexes.

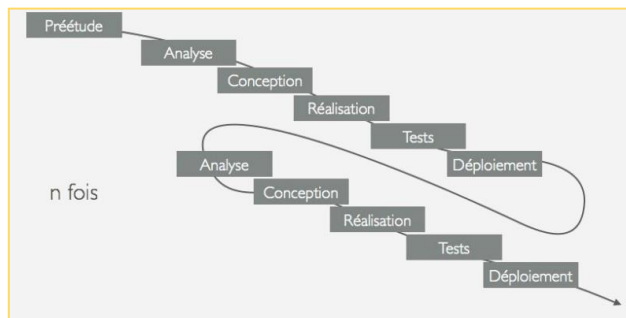
- **Définition**

- Procédé qui a pour objectif de permettre de formaliser les étapes du développement d'un système afin de rendre ce développement plus fidèle aux besoins du client.

- **Méthodologie : Cascade**



- **Méthodologie : Cyclique**



- **Méthodologie agile : SCRUM**

La méthodologie Scrum est un cadre de gestion de projet agile qui vise à fournir des produits logiciels de haute qualité de manière itérative et incrémentale. Voici un résumé des principaux concepts de Scrum.

- Équipes Auto-organisées

Les équipes Scrum sont auto-organisées, ce qui signifie qu'elles sont responsables de la planification, de l'exécution et de l'amélioration de leur travail.

- Sprint

Les projets Scrum sont divisés en itérations appelées "Sprints", qui ont généralement une durée de 2 à 4 semaines. À la fin de chaque Sprint, une version du produit fonctionnelle est livrée.

METHODOLOGIE

(Technique)

DESIGN PATTERN MVP.

- **Scrum Master**

Assure la bonne application des pratiques Scrum, Protège l'équipe, assiste le PO, pas d'autorité hiérarchique.

- Sprint Review

Les développeurs démontrent le résultat du Sprint au stakeholder ou au PO.

- Retrospective

Bilan sur le déroulement du sprint afin d'améliorer l'équipe